



ŽILINSKÁ UNIVERZITA V ŽILINE

Fakulta riadenia
a informatiky

Semestrálna práca z predmetu
Algoritmy a údajové štruktúry 2

Vypracoval: Bc. Richard Vymazal

Študijná skupina: 5ZIA12

cvičiaci: Ing. Peter Jankovič, PhD.

Termín cvičenia: utorok bloky 12 – 13

v Žiline dňa 11.11.2



Obsah

Úvod	3
Abstraktná trieda Data – rozhranie pre dátové typy	3
Abstraktná trieda Tree – rozhranie pre binárne stromové štruktúry	4
Trieda BinaryTree – binárna stromová štruktúra	5
Trieda BalancedTree – vyvažovaná binárna stromová štruktúra	5
Implementácia rotácií	5
Výsledky testov	6
Porovnanie AVL a TreeSet	7
Testovanie základných operácií v stromových štruktúrach	7
Komplexný test stromu	7
Test vkladania (Insert)	8
Test mazania (Delete)	8
Test vyhľadávania (Find)	8
Testy hľadania minima/maxima	8
Test rozsahového vyhľadávania	9
Trieda PatientData – reprezentácia pacienta a jeho testov	9
Hlavné vlastnosti a funkcionality	9
Trieda PcrTestData – reprezentácia PCR testu	10
Hlavné vlastnosti a funkcionality	10
Trieda LocationData – reprezentácia lokality a jej PCR testov	10
Hlavné vlastnosti a funkcionality	10
Trieda WorkplaceData – reprezentácia pracoviska a jeho PCR testov	11
Hlavné vlastnosti a funkcionality	11
Trieda SortedData – pomocný dátový typ pre zoradenie okresov a regiónov	11
Trieda DemoSystem – centrálna správa dát a analytika testov	12
Architektúra a vnútorné dáta	12
Hlavná funkcionality	14
Import a export dát	17
Trieda MainFrame – GUI	17
Princíp oddelenia dátovej logiky od GUI	17
Hlavné funkcionality rozhrania	17
Použité komponenty	18
UML diagram tried	19
Využitie AI v projekte	20



Úvod

Tento projekt sa zameriava na implementáciu a testovanie **binárnych stromových štruktúr** v programovacom jazyku **Java**.

Obsahuje návrh a realizáciu všeobecného rozhrania pre prácu s dátami a stromami, ako aj konkrétne implementácie **binárneho vyhľadávacieho stromu (BinaryTree)** a **samo-vyvažovacieho stromu (BalancedTree)** založeného na princípoch AVL algoritmu.

Projekt ďalej demonštruje praktické využitie stromov v reálnom kontexte správy pacientov a PCR testov. Súčasťou aplikácie je aplikačná logika, ktorá umožňuje:

- vkladanie a mazanie pacientov a testov s kontrolou konzistencie údajov (napr. formát rodného čísla);
- pokročilé vyhľadávanie podľa dátumových intervalov, krajov, okresov či pracovísk;
- agregované štatistiky a prehľady, ako napríklad počet chorých pacientov alebo zoradené štatistiky podľa krajov/okresov;
- import a export dát vo formáte CSV;
- generovanie testovacích dát.

Na vizualizáciu a interaktívnu prácu so systémom je k dispozícii jednoduché **Swing GUI**, ktoré umožňuje rýchle vyhľadávanie, zadávanie údajov cez formuláre vrátane výberu dátumu/času pomocou JSpinner a prehľadné zobrazenie výsledkov.

Z hľadiska testovania projekt obsahuje formy testov pre základné operácie stromov (insert, find, delete, vrátane rotácií AVL a operácie rangeFind).

Celkovo projekt demonštruje možnosti využitia generických stromových dátových štruktúr v reálnych aplikáciách, pričom kombinuje implementáciu dátových štruktúr s praktickým využitím v oblasti správy pacientov a testov.

Abstraktná trieda Data – rozhranie pre dátové typy

Z dôvodu zachovania flexibility pri práci s rôznymi dátovými typmi v projekte sú všetky dátové triedy odvodené od abstraktnej triedy **Data**.

Táto trieda definuje abstraktnú metódu **compare**, ktorá umožňuje porovnávanie hodnôt jednotlivých dát pri vkladaní prvkov do **binárnych stromov** (BST, AVL).

Keďže ide o abstraktnú triedu, konkrétnu implementáciu metódy **compare** si definuje každý dátový typ samostatne podľa vlastnej potreby. Tým sa zabezpečí, že binárne štruktúry môžu fungovať univerzálne — bez ohľadu na to, aký typ dát spracúvajú.

Okrem porovnávacej metódy **compare** obsahuje trieda **Data** aj ďalšie abstraktné metódy:

- **getKey()** – poskytuje kľúčový identifikátor dátového objektu, napríklad unikátne ID pacienta alebo testu.
- **formatToCsv()** – zabezpečuje konvertovanie dátového obsahu do CSV formátu, čo uľahčuje implementáciu exportu a dávkové spracovanie údajov.



- **formatData()** – poskytuje textovú reprezentáciu dát vhodnú na zobrazovanie v GUI.

Trieda Data zároveň implementuje rozhranie **Comparable**, vďaka čomu sú dátové typy kompatibilné s triediacimi algoritmami a dátovými štruktúrami, ktoré porovnanie vyžadujú. Táto funkcionálnosť sa v projekte využíva len pri **triedení pomocných dátových štruktúr počas testovania**.

Abstraktná trieda Tree – rozhranie pre binárne stromové štruktúry

Abstraktná trieda Tree predstavuje základné rozhranie pre všetky binárne stromové štruktúry v projekte, vrátane klasického **BinaryTree (BST)** a samo-vyvažovacieho **BalancedTree (AVL)**. Jej účelom je poskytnúť spoločnú kostru stromu, základné operácie a pomocné metódy, ktoré môžu využívať rôzne implementácie stromov.

Trieda Tree je generická a pracuje s typom T, ktorý musí byť odvodený od abstraktnej triedy Data. To umožňuje, aby strom spracovával rôzne typy dát – napríklad pacientov, PCR testy alebo pracoviská – a zároveň poskytuje jednotné porovnávacie mechanizmy.

Každý uzol stromu je reprezentovaný vnútornou triedou **Node**, ktorá obsahuje referencie na ľavého a pravého potomka, uložené dáta a výšku uzla. Výška uzla, ktorá sa využíva pri kontrole vyváženosti, je dôležitá najmä pri implementácii AVL stromu. Trieda Node poskytuje metódy na aktualizáciu výšky a kontrolu rovnováhy stromu.

Porovnávanie prvkov môže byť realizované dvoma spôsobmi: prirodzené porovnanie cez metódu **compare** z triedy **Data** alebo pomocou voliteľného **Comparatoru**, ktorý **môže** byť poskytnutý pri vytváraní stromu.

Abstraktná trieda Tree definuje základné operácie pre prácu so stromom, vrátane:

- **vkladania (insert) a mazania (delete) prvkov**, pričom konkrétna implementácia je realizovaná v odvodených stromoch,
- **vyhľadávania konkrétnych prvkov (find)**, iteratívne prechádza strom od koreňa a hľadá uzol s danou hodnotou, pri nájdení uzla vracia jeho dáta,
- **vyhľadávania prvkov v rozsahu (rangeFind)**, vyhľadáva všetky prvky v danom rozsahu, pričom používa zásokník a iteratívny in-order prechod kde nenavštevuje podstromy, ktoré nemôžu obsahovať hodnoty z rozsahu,
- **nájdenia minima a maxima (findMin/findMax)** v celom strome alebo v podstrome začínajúcom konkrétnym uzlom, pomocou iteratívneho prechodu len ľavých alebo pravých potomkov,
- **nájdenie alebo vkladanie (findOrInsert)**, ktorá vráti hľadaný prvok ak v strome existuje a ak nie, tak prvok vloží,
- **prechodov stromom (inOrder/levelOrder)** – in-order pre získanie zoradeného zoznamu dát a level-order pre spracovanie uzlov po úrovniach, kde metódy prechádzajú strom iteratívne a vracajú **iterátor**,
- **kontroly prázdnoty stromu**, vracia informáciu o tom či strom má koreň alebo nie.

Táto štruktúra umožňuje jednotný prístup k stromom, jednoduché rozšírenie o nové algoritmy alebo typy stromov a zároveň zaručuje, že operácie nad stromom zostávajú konzistentné.

Trieda BinaryTree – binárna stromová štruktúra

Trieda predstavuje konkrétnu implementáciu **binárneho vyhľadávacieho stromu (BST)**, ktorá nadväzuje na abstraktnú triedu Tree.

Zabezpečuje základnú funkcionálnosť binárneho stromu, v ktorom je každý uzol usporiadaný podľa hodnôt svojich dát — všetky prvky v ľavom podstrome sú menšie ako hodnota v koreňovom uzle a všetky v pravom podstrome sú väčšie.

Trieda implementuje dve operácie:

- **insert(T data)** – vkladanie nového prvku do stromu.
Metóda prechádza strom od koreňa a hľadá správne miesto, kam môže byť nový uzol vložený tak, aby bola zachovaná BST vlastnosť.
V prípade, že hodnota už v strome existuje, **nevloží** sa duplicitne.
- **delete(T data)** – odstraňovanie prvku zo stromu tak. Metóda začína od koreňa a hľadá prvok na odstránenie. Po jeho nájdení môžu nastať 3 rôzne spôsoby mazania na základe počtu potomkov prvku v strome. Každý spôsob zabezpečí požadované odstránenie prvku zo stromu a taktiež zachovanie BST vlastností.

Trieda BalancedTree – vyvažovaná binárna stromová štruktúra

Trieda predstavuje implementáciu **samo-vyvažovacieho binárneho stromu**, založenú na princípoch **AVL stromu**.

Je odvodená od abstraktnej triedy Tree a rozširuje jej funkcionálnosť o automatické udržiavanie **rovnováhy výšky stromu** po každej operácii vkladania alebo mazania prvku.

Základným cieľom tejto implementácie je zabezpečiť, aby bol strom **vždy čo najviac vyvážený**, a tým sa minimalizoval čas potrebný na vyhľadávanie, vkladanie či mazanie prvkov. Vďaka tomu má strom garantovanú **logaritmickú časovú zložitosť $O(\log n)$** pre väčšinu operácií.

Trieda implementuje dve hlavné operácie:

- **insert(T data)** – vloží nový uzol do stromu podľa BST pravidiel a následne spätne prechádza cestu ku koreňu, pričom aktualizuje výšky uzlov a vyhodnocuje **balančný faktor**. Ak je zistená nerovnováha, aplikuje príslušnú rotáciu (jednoduchú alebo dvojité), aby sa štruktúra stromu opäť vyvážila. Spätne prechádzanie uzlov môže byť skoršie ukončené v prípade, že nerovnováha nie je zistená.
- **delete(T data)** – odstráni uzol z BST a po vykonaní operácie vykoná rovnaký proces spätnej aktualizácie a vyvažovania ako bolo popísané pri operácii insert.

Pri prechádzaní stromu a rekonštrukcii jeho častí po úpravách sa využíva štruktúra **Deque** (ako zásobník), ktorá umožňuje efektívne iteratívne spracovanie **bez** potreby rekúzie.

Implementácia rotácií

Rotácie sú **lokálne** transformácie podstromu, ktoré zachovávajú BST vlastnosť (poradie prvkov) a zároveň menia jeho tvar tak, aby sa znížila výšková nerovnováha.

Implementované sú štyri typy rotácií: **jednoduché** (ľavá a pravá) a **dvojité** (ľavo-pravá a pravo-ľavá). Jednoduché rotácie riešia priame nerovnováhy, zatiaľ čo dvojité kombinujú dve jednoduché rotácie



pre prípad nepriamych nerovnováh. Po každej rotácii sa aktualizujú výšky dotknutých uzlov a novým koreňom podstromu sa stáva uzol, ktorý po rotácii zaujme vyváženú pozíciu.

Výsledky testov

V rámci projektu bolo vykonané **porovnanie výkonnosti implementovaných stromových štruktúr – binárneho vyhľadávacieho stromu (BST) a samo-vyvažovacieho AVL stromu.**

Cieľom testov bolo zmerať časové nároky základných operácií – **vloženie (insert)**, **odstránenie (delete)** a **vyhľadanie (find)** – pri spracovaní veľkého množstva dát.

Každý test bol vykonaný na **10 miliónoch iterácií**, pričom výsledky predstavujú priemerné časy v **mikrosekundách**.

Štruktúra	Vkladanie	Rozdiel	Mazanie	Rozdiel	Vyhľadávanie	Rozdiel
BST	0.090	-	0.089	-	0.082	-
AVL	0.123	+36.7%	0.123	+38.2%	0.071	-13.4%

Výsledok výkonnostného testu BST a AVL

Výsledky ukazujú, že operácie **vloženia** a **odstránenia** sú v prípade AVL stromu mierne pomalšie než v klasickom BST. Tento rozdiel je spôsobený dodatočnými výpočtami pri **udržiavaní rovnováhy stromu** – aktualizáciou výšok uzlov a vykonávaním rotácií. Na druhej strane, operácia **vyhľadávania** (find) je v AVL strome o niečo rýchlejšia, pretože strom zostáva **vyvážený**, čím sa znižuje jeho priemerná hĺbka a tým aj počet porovnaní potrebných na nájdenie prvku.

Z výsledkov teda vyplýva, že AVL strom je vhodnejší pre **časté vyhľadávanie** v rozsiahlych dátových množinách, zatiaľ čo jednoduchý BST môže byť efektívnejší pri **sekvenčnom vkladaní a mazaní**, ak nehrozí jeho výrazné rozváženie.

V ďalšom teste bola analyzovaná výkonnosť štruktúr pri špecifickom type vstupných dát – **inkrementujúcich sa kľúčoch**. Tento spôsob vkladania má zásadný vplyv na správanie binárneho vyhľadávacieho stromu (BST), pretože pri absencii vyvažovania sa strom **zdegeneruje na lineárny zoznam**. V dôsledku toho dochádza k výraznému zhoršeniu časovej zložitosti operácií z logaritmickkej na lineárnu.

Test bol vykonaný na **50 000 iteráciách**, pričom výsledky predstavujú celkové časy v **sekundách**.

Kľúče	Vkladanie	Rozdiel
Náhodné	0.011	-
Inkrementujúce	1.692	+15 272.7%

Výsledok inkrementačného testu BST

Z výsledkov je zrejmé, že pri vkladaní inkrementujúcich sa kľúčov sa výkon BST dramaticky zhoršuje – čas vkladania narastá viac než **150-násobne**. Tento jav presne odráža teoretické správanie nevyváženého stromu, ktorý sa pri sekvenčnom vkladaní mení na lineárny zoznam, čím sa každé ďalšie vloženie predlžuje úmerne s počtom prvkov.



Porovnanie AVL a TreeSet

TreeSet predstavuje v Java **štandardnú implementáciu vyvažovaného stromu** postaveného na **Red-Black Tree (RB tree)**, ktorý je funkčne podobný AVL stromu, no používa odlišnú stratégiu vyvažovania. Porovnanie vlastnej implementácie AVL stromu s TreeSet umožňuje zhodnotiť, ako sa vlastná implementácia správa v porovnaní s optimalizovanou implementáciou.

Každý test bol vykonaný na **10 miliónoch iterácií**, pričom výsledky predstavujú priemerné časy v **mikrosekundách**.

Štruktúra	Vkladanie	Rozdiel	Mazanie	Rozdiel	Vyhľadávanie	Rozdiel
TreeSet	0.078	-	0.076	-	0.054	-
AVL	0.107	+37.2%	0.108	+42.1%	0.060	+11.1%

Výsledok výkonnostného testu TreeSet a AVL

Z výsledkov vyplýva, že **TreeSet** dosahuje vo všetkých testovaných operáciách nižšie priemerné časy, čo je očakávané vzhľadom na jeho vysoký stupeň optimalizácie a dlhoročné ladenie v štandardnej knižnici Javy.

Napriek tomu je výkon vlastnej implementácie **AVL stromu** veľmi stabilný a pomerne blízky výkonu **TreeSet**, čo potvrdzuje správnosť a efektívnosť implementácie základných stromových operácií.

Testovanie základných operácií v stromových štruktúrach

Cieľom je overiť **výkonnosť, stabilitu a korektnosť implementácie** pri veľkom počte operácií:

Komplexný test stromu

Metóda **basicTest** vykonáva **kombinovaný test** stromovej štruktúry, ktorý simuluje reálne používanie dátovej štruktúry.

Počas testu sa v náhodnom poradí vykonávajú operácie **vkładania (insert)**, **mazania (delete)**, **vyhľadávania (find)** a **rozsahového vyhľadávania (rangeFind)**.

Existuje aj metóda **weightedTest**, ktorá vykonáva operácie s percentuálnou pravdepodobnosťou.

Charakteristika testu:

- Počet iterácií: **10 miliónov**
- Poradie operácií je generované náhodne
- Pravidelne sa kontroluje **in-order správnosť** (zoradenie prvkov)
- Pri AVL strome sa navyše pravidelne overuje **správnosť výšky uzlov a vyváženosť**

Cieľ: Overiť dlhodobú **stabilitu a konzistentnosť implementácie** počas náhodných sekvencií operácií.

(Tento test neuvádza konkrétne časové výsledky – ide o funkčný test správnosti.)



Test vkladania (Insert)

Táto metóda testuje **výkonnosť a korektnosť** operácie vloženia uzlov do stromu.

Postupne sa vkladá **10 miliónov náhodne generovaných hodnôt**, pričom sa meria **celkový čas vykonania**.

Charakteristika testu:

- Každý vkladací prvok je jedinečný (pri duplicite sa vkladanie opakuje),
- Vložené hodnoty sa ukladajú do poľa pre neskoršie testy (delete, find, ...).

Štruktúra	Čas vkladania (s)
BST	12.29
AVL	15.04

Test mazania (Delete)

Test hodnotí **efektivitu a správnosť operácie odstránenia uzlov**. Zo stromu sa postupne odstraňuje **2 milióny prvkov**, ktoré boli predtým vložené počas testu vkladania.

Charakteristika testu:

- Každé odstránenie sa overuje – kontroluje sa, či bol uzol skutočne vymazaný.

Štruktúra	Čas mazania (s)
BST	2.62
AVL	3.45

Test vyhľadávania (Find)

Metóda testuje **presnosť a rýchlosť vyhľadávania** existujúcich uzlov v strome.

Vyhľadávanie prebieha na vzorke 5 miliónov uzlov, ktoré boli predtým vložené.

Charakteristika testu:

- Overuje sa, že každý vyhľadaný prvok v strome skutočne existuje.

Štruktúra	Čas vyhľadania (s)
BST	8.69
AVL	6.68

Testy hľadania minima/maxima

Dané testy overujú **správnosť a výkon operácií** na nájdenie minimálnej a maximálnej hodnoty v strome alebo podstrome pre **2 milióny prvkov**.

Charakteristika testu:

- Test sa vykonáva na množine náhodne vybraných uzlov



Štruktúra	Čas vyhľadania (s)
BST	3.82/3.84
AVL	2.88/2.90

Test rozsahového vyhľadávania

Metóda testuje funkčnosť a efektivitu operácie, ktorá vracia **všetky uzly nachádzajúce sa v danom intervale hodnôt** pre 1 milión intervalov.

Charakteristika testu:

- Prvky v pomocnom poli sa usporiadajú
- Definujú sa intervaly <min, max> a vykonáva sa rozsahové vyhľadanie
- Overuje sa, že každý vrátený prvok skutočne patrí do daného intervalu

Štruktúra	Čas vyhľadania (s)
BST	8.78
AVL	4.65

Tieto testy umožňujú komplexné hodnotenie správania a výkonu BST a AVL stromov. Získané časy pre jednotlivé operácie poskytujú prehľad o tom, v ktorých typoch operácií vyvážená štruktúra AVL prináša výhodu (napr. pri vyhľadávaní a rozsahových dotazoch) a kde naopak jednoduchší BST dosahuje rýchlejšie výsledky.

Trieda PatientData – reprezentácia pacienta a jeho testov

Trieda PatientData predstavuje konkrétny dátový typ odvodený od abstraktnej triedy Data a slúži na **uchovávanie informácií o pacientovi** spolu s jeho PCR testami. Každý pacient je jednoznačne identifikovaný unikátnym **ID**, ktoré slúži ako kľúč pre porovnávanie a vyhľadávanie v stromových štruktúrach.

Hlavné vlastnosti a funkcionality

- **Základné údaje o pacientovi**
Trieda uchováva rodné číslo, meno, priezvisko a dátum narodenia pacienta.
- **Správa PCR testov**
Každý pacient obsahuje dva binárne stromy:
 - record – základný BST pre testy, zoradený podľa kľúča testu,
 - recordByDate – AVL zoradený podľa dátumu testu a potom podľa kľúča, ktorý umožňuje efektívne vyhľadávanie testov v časových intervaloch.

Metódy addTest() a removeTest() umožňujú pridávať a odstraňovať testy z oboch stromov súčasne.



- **Porovnávanie pacientov**
Implementovaná metóda **compare** porovnáva pacientov podľa ich unikátneho ID/Rodného čísla.
- **Export a zobrazenie dát**
 - `formatToCsv()` – formátuje údaje pacienta do CSV formátu vhodného na export alebo dávkové spracovanie.
 - `formatData()` – poskytuje prehľadnú textovú reprezentáciu pacienta vrátane jeho mena, priezviska a dátumu narodenia, vhodnú pre zobrazenie v GUI.

Trieda `PcrTestData` – reprezentácia PCR testu

Trieda `PcrTestData` predstavuje konkrétny dátový typ odvodený od abstraktnej triedy `Data` a slúži na **uchovávanie informácií o PCR testoch pacientov**. Každý test je jednoznačne identifikovaný unikátnym celočíselným **ID**, ktoré slúži ako kľúč pre porovnávanie a vyhľadávanie v stromových štruktúrach.

Hlavné vlastnosti a funkcionality

- **Základné údaje o teste**
Trieda uchováva rodné číslo pacienta ktorému daný test patrí, dátum a čas vykonania testu (`LocalDateTime`), pracovisko, okres a kraj, ako aj výsledok a hodnotu testu. Pre doplňujúce informácie sú k dispozícii aj textové poznámky.
- **Porovnávanie**
Implementovaná metóda **compare** porovnáva testy podľa ich unikátneho ID.
- **Export a zobrazenie dát**
 - `formatToCsv()` – formátuje údaje testu do CSV formátu, vhodného na export alebo dávkové spracovanie. Pri exporte sa zabezpečuje, že textové poznámky neobsahujú znaky, ktoré by mohli narušiť formát CSV.
 - `formatData()` – poskytuje prehľadnú textovú reprezentáciu testu vrátane všetkých dôležitých informácií, vhodnú pre zobrazenie v GUI alebo logovaní.

Trieda `LocationData` – reprezentácia lokality a jej PCR testov

Trieda `LocationData` predstavuje konkrétny dátový typ odvodený od abstraktnej triedy `Data` a slúži na **uchovávanie informácií o konkrétnej geografickej lokalite** vrátane PCR testov vykonaných v danej oblasti. Každá lokalita je jednoznačne identifikovaná číselným **kódom**, ktorý slúži ako kľúč pre porovnávanie a vyhľadávanie v stromových štruktúrach.

Hlavné vlastnosti a funkcionality

- **Správa testov v lokalite**
Každá lokalita obsahuje dve stromové štruktúry pre PCR testy:
 - `recordByDate` – AVL zoradený podľa dátumu testu a potom podľa ID, vhodný na vyhľadávanie testov v časových intervaloch,



- positiveRecordByDate – AVL obsahujúci len pozitívne testy, zoradený podľa dátumu a potom podľa kľúča.
- **Pridávanie a odstraňovanie testov**
Metódy addTest() a removeTest() zabezpečujú súčasnú úpravu všetkých stromov.
- **Porovnávanie**
Implementovaná metóda **compare** porovnáva lokality podľa ich číselného kódu.

Trieda LocationData slúži v systéme na efektívne ukladanie testov pre jednotlivé **kraje a okresy**, pričom oba typy lokalít môžu byť reprezentované jedným dátovým typom. Pri spracovaní sa zohľadňuje hierarchická závislosť kódov, kde každý kód okresu vždy patrí len jednému kraju.

Trieda WorkplaceData – reprezentácia pracoviska a jeho PCR testov

Trieda WorkplaceData predstavuje konkrétny dátový typ odvodený od abstraktnej triedy Data a slúži na **uchovávanie informácií o pracovisku** vrátane PCR testov vykonaných na danom pracovisku. Každé pracovisko je jednoznačne identifikované číselným **ID**, ktoré slúži ako kľúč pre porovnávanie a vyhľadávanie v stromových štruktúrach.

Hlavné vlastnosti a funkcionality

- **Správa testov na pracovisku**
Každé pracovisko obsahuje jeden binárny strom:
 - recordByDate – AVL zoradený podľa dátumu testu a potom podľa kľúča testu, vhodný na vyhľadávanie testov v časových intervaloch.
- **Pridávanie a odstraňovanie testov**
Metódy addTest() a removeTest() zabezpečujú úpravu stromu pri pridávaní alebo odstraňovaní testov.
- **Porovnávanie a kľúč pracoviska**
Implementovaná metóda **compare** porovnáva pracoviská podľa ich ID.

Trieda SortedData – pomocný dátový typ pre zoradenie okresov a regiónov

Trieda SortedData predstavuje **pomocný dátový typ** odvodený od abstraktnej triedy Data, ktorý sa používa výhradne pre **zoraďovanie geografických jednotiek podľa počtu chorých pacientov** v danom časovom období. Tento typ dát umožňuje jednoduché vloženie do binárnych stromov a efektívne získanie zoradeného výstupu.

Slúži teda ako **dočasná dátová štruktúra** pre analytické spracovanie a zobrazovanie informácií

Trieda DemoSystem – centrálna správa dát a analytika testov

Trieda DemoSystem predstavuje **jadro systému** pre správu údajov o pacientoch a PCR. Ide o hlavnú triedu, ktorá spája dátové modely, udržiava konzistentné prepojenia medzi nimi a vykonáva rôzne dotazy nad nimi.

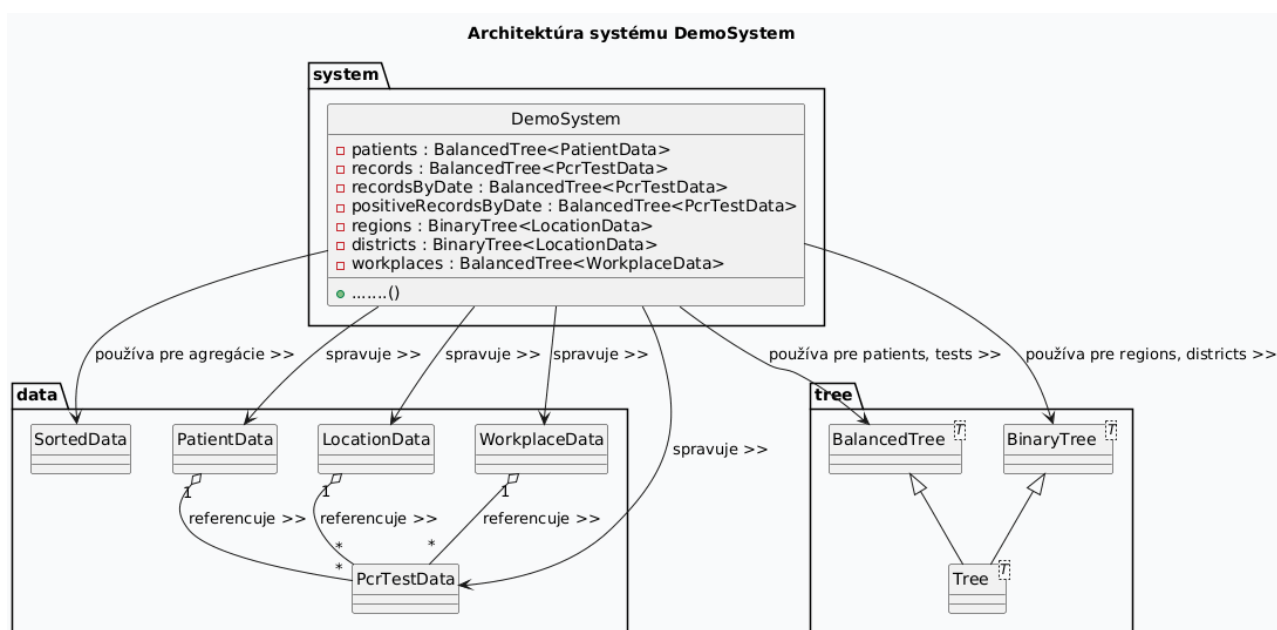
Je navrhnutá tak, aby poskytovala efektívne operácie nad veľkým objemom dát vďaka využitiu **vyvážených stromov (BalancedTree)** a **binárnych stromov (BinaryTree)**, ktoré zabezpečujú logaritmickú časovú zložitosť väčšiny operácií.

Architektúra a vnútorné dáta

Trieda uchováva všetky entity systému vo forme stromových dátových štruktúr:

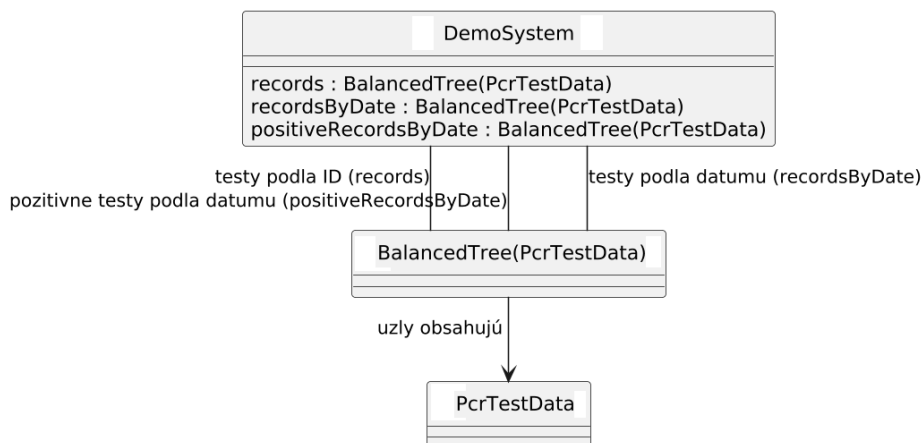
- **Pacienti** (BalancedTree<PatientData> patients) – centrálna úložisko pacientov, kde je každý pacient identifikovaný jedinečným ID.
- **Testy** (BalancedTree<PcrTestData> records) – hlavný strom PCR testov, indexovaný podľa ID testu.
- **Testy podľa dátumu** (BalancedTree <PcrTestData> recordsByDate) – pomocná štruktúra umožňujúca vyhľadávanie časových intervalov, indexovaná podľa času testov a potom podľa ich ID.
- **Pozitívne testy podľa dátumu** (BalancedTree <PcrTestData> positiveRecordsByDate) – pomocná štruktúra uchováajúca len testy s pozitívnym výsledkom.
- **Okresy a regióny** (BinaryTree<LocationData> districts, BinaryTree<LocationData> regions) – stromy reprezentujúce jednotlivé geografické jednotky.
- **Pracoviská** (BalancedTree <WorkplaceData> workplaces) – strom pracovísk, ktoré vykonávajú testy.

Táto organizácia dát umožňuje, že **jedna** inštancia je uchovávaná vo viacerých stromových štruktúrach súčasne, čo umožňuje rýchle dotazy z rôznych perspektív bez nutnosti lineárneho prehľadávania.

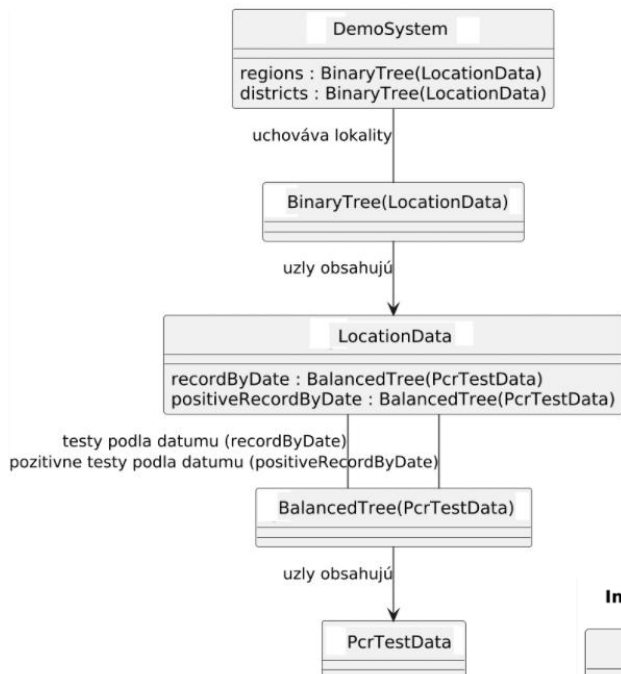




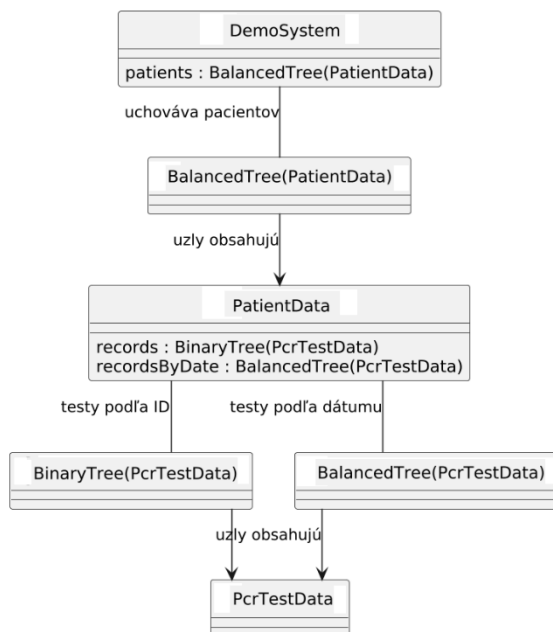
Implementácia AVL pre ukladanie testov



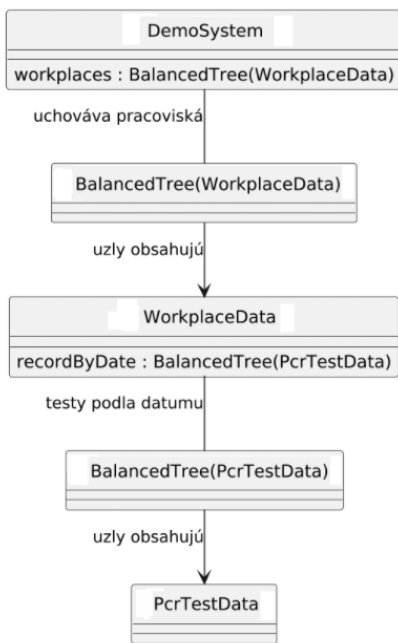
Implementácia BST pre okresy/kraje



Implementácia AVL pre ukladanie pacientov



Implementácia AVL pre pracoviská





Hlavná funkcionálna

Systém poskytuje všetkých 21 požadovaných operácií, ktoré sú neskôr sprístupnené na GUI. Zároveň je zabezpečená dátová enkapsulácia, takže na GUI sa vždy dostáva iba obsah dát vo forme reťazcov, a nikdy samotné interné dátové objekty.

1. Vloženie výsledku PCR testu do systému (**insertTest**)

$O(\log n + \log t + \log p + \log nt + (\log r + \log tr + \log trp) + (\log d + \log td + \log tdp) + (\log w + \log tw))$ kde **n** - počet pacientov, **t** - počet všetkých testov, **p** - počet pozitívnych testov, **nt** - počet testov konkrétného pacienta, **r** - počet krajov, **tr** - počet testov v kraji, **trp** - počet pozitívnych testov v kraji, **d** - počet okresov, **td** - počet testov v okrese, **tdp** - počet pozitívnych testov v okrese, **w** - počet pracovísk, **tw** - počet testov na pracovisku.

2. Vyhľadanie výsledku testu pre pacienta a zobrazenie všetkých údajov (**findTestForPatient**)

$O(\log p + \log t)$ kde **p** - počet pacientov, **t** - počet PCR testov konkrétného pacienta.

3. Výpis všetkých uskutočnených PCR testov pre daného pacienta usporiadaných podľa dátumu a času ich vykonania (**findAllTestsForPatient**)

$O(\log p + t)$ kde **p** - počet pacientov, **t** - počet PCR testov konkrétného pacienta.

4. Výpis všetkých pozitívnych testov uskutočnených za zadané časové obdobie pre zadaný okres (**findAllPositiveTestsForDistrict**)

$O(\log d + \log t + (k \cdot \log p))$ kde **d** - počet okresov, **t** - počet pozitívnych testov v okrese, **p** - počet pacientov, **k** - počet pozitívnych testov okresu v zadanom časovom období.

5. Výpis všetkých testov uskutočnených za zadané časové obdobie pre zadaný okres (**findAllTestsForDistrict**)

$O(\log d + \log t + (k \cdot \log p))$ kde **d** - počet okresov, **t** - počet testov okresu, **p** - počet pacientov, **k** - počet testov okresu v zadanom časovom období.

6. Výpis všetkých pozitívnych testov uskutočnených za zadané časové obdobie pre zadaný kraj (**findAllPositiveTestsForRegion**)

$O(\log r + \log t + (k \cdot \log p))$ kde **r** - počet krajov, **t** - počet pozitívnych testov kraja, **p** - počet pacientov, **k** - počet pozitívnych testov kraja v zadanom časovom období.

7. Výpis všetkých testov uskutočnených za zadané časové obdobie pre zadaný kraj (**findAllTestsForRegion**)

$O(\log r + \log t + (k \cdot \log p))$ kde **r** - počet krajov, **t** - počet testov kraja, **p** - počet pacientov, **k** - počet testov kraja v zadanom časovom období.



8. Výpis všetkých pozitívnych testov uskutočnených za zadané časové obdobie (**findAllPositiveTests**)

$O(\log t + (k \cdot \log p))$ kde t – počet testov, p – počet pacientov, k – počet pozitívnych testov v zadanom časovom období.

9. Výpis všetkých testov uskutočnených za zadané časové obdobie (**findAllTests**)

$O(\log t + (k \cdot \log p))$ kde t – počet testov, p – počet pacientov, k – počet testov v zadanom časovom období.

10. Výpis chorých osôb v okrese k zadanému dátumu, pričom osobu považujeme za chorú X dní od pozitívneho testu (**findAllSickPatientsForDistrict**)

$O(\log d + \log t + (k \cdot \log k) + (k \cdot \log p))$ kde d – počet okresov, t – počet pozitívnych testov okresu, p – počet pacientov, k – počet pozitívnych testov okresu v zadanom časovom období.

11. Výpis chorých osôb v okrese k zadanému dátumu, pričom osobu považujeme za chorú X dní od pozitívneho testu, pričom choré osoby sú usporiadané podľa hodnoty testu (**findAllSortedSickPatientsForDistrict**)

$O(\log d + \log t + (k \cdot \log k) + (k \cdot \log p))$ kde d – počet okresov, t – počet pozitívnych testov okresu, p – počet pacientov, k – počet pozitívnych testov okresu v zadanom časovom období.

12. Výpis chorých osôb v kraji k zadanému dátumu, pričom osobu považujeme za chorú X dní od pozitívneho testu (**findAllSickPatientsForRegion**)

$O(\log r + \log t + (k \cdot \log k) + (k \cdot \log p))$ kde r – počet krajov, t – počet pozitívnych testov kraja, p – počet pacientov, k – počet pozitívnych testov kraja v zadanom časovom období.

13. Výpis chorých osôb k zadanému dátumu, pričom osobu považujeme za chorú X dní od pozitívneho testu (**findAllSickPatients**)

$O(\log t + (k \cdot \log k) + (k \cdot \log p))$ kde t – počet pozitívnych testov, p – počet pacientov, k – počet pozitívnych testov v zadanom časovom období.

14. Výpis jednej chorej osoby k zadanému dátumu, pričom osobu považujeme za chorú X dní od pozitívneho testu z každého okresu, ktorá má najvyššiu hodnotu testu (**findMostSickPatientForDistrict**)

$O(d \cdot (\log t + k \log p))$ kde d – počet okresov, t – počet pozitívnych testov okresu, p – počet pacientov, k – počet pozitívnych testov okresu v zadanom časovom období.



15. Výpis okresov usporiadaných podľa počtu chorých osôb k zadanému dátumu, pričom osobu považujeme za chorú X dní od pozitívneho testu (**findSortedDistrictsBySickPatients**)

$O(d \cdot (\log t + k \cdot \log k + k \cdot \log p) + d \cdot \log d)$ kde **d** – počet okresov, **t** – počet pozitívnych testov okresu, **p** – počet pacientov, **k** – počet pozitívnych testov okresu v zadanom časovom období.

16. Výpis krajov usporiadaných podľa počtu chorých osôb k zadanému dátumu, pričom osobu považujeme za chorú X dní od pozitívneho test (**findSortedRegionsBySickPatients**)

$O(r \cdot (\log t + k \cdot \log k + k \cdot \log p) + r \cdot \log r)$ kde **r** – počet krajov, **t** – počet pozitívnych testov kraja, **p** – počet pacientov, **k** – počet pozitívnych testov kraja v zadanom časovom období.

17. Výpis všetkých testov uskutočnených za zadané časové obdobie na danom pracovisku (**findAllTestsForWorkplace**)

$O(\log w + \log t + (k \cdot \log p))$ kde **w** – počet pracovísk, **t** – počet testov pracoviska, **p** – počet pacientov, **k** – počet testov pracoviska v zadanom časovom období.

18. Vyhľadanie PCR testu podľa jeho kódu (**findTest**)

$O(\log t)$ kde **t** – počet testov.

19. Vloženie osoby do systému (**insertPatient**)

$O(\log p)$ kde **p** – počet pacientov.

20. Trvalé a nevratné vymazanie výsledku PCR testu, test je definovaných svojim kódom (**deleteTest**)

$O(\log t + \log p + (\log r + \log tr + \log trp) + (\log d + \log td + \log tdp) + (\log w + \log tw) + (\log n + \log nt))$ kde **t** – počet testov, **p** – počet pozitívnych testov, **r** – počet krajov, **tr** – počet testov kraja, **trp** – počet pozitívnych testov kraja, **d** – počet okresov, **td** – počet testov okresu, **tdp** – počet pozitívnych testov okresu, **w** – počet pracovísk, **tw** – počet testov pracoviska, **n** – počet pacientov, **nt** – počet testov konkrétneho pacienta.

21. Vymazanie osoby zo systému aj s jej výsledkami PCR testov (**deletePatient**)

$O(\log n + k \cdot (\log t + \log p + (\log r + \log tr + \log trp) + (\log d + \log td + \log tdp) + (\log w + \log tw)))$ kde **n** – počet pacientov, **k** – počet testov pacienta, **t** – počet testov, **p** – počet pozitívnych testov, **r** – počet krajov, **tr** – počet testov kraja, **trp** – počet pozitívnych testov kraja, **d** – počet okresov, **td** – počet testov okresu, **tdp** – počet pozitívnych testov okresu, **w** – počet pracovísk, **tw** – počet testov pracoviska.

Import a export dát

- **Načítanie dát** – loadPatients, loadTests načítajú CSV súbory a postupne vkladajú jednotlivé objekty do stromov pomocou insetPatient a insertTest ($O(p/t * O(\text{insetPatient}/\text{insertTest}))$).
- **Uloženie dát** – savePatients, saveTests vykonávajú **level-order** prechod stromu a zapisujú údaje do súboru ($O(p/t)$).

(*p* – počet pacientov, *t* – počet testov)

Trieda MainFrame – GUI

Trieda **MainFrame** predstavuje **hlavné grafické používateľské rozhranie** (GUI) pre systém PCR testov, postavené na knižnici Swing. Slúži ako centrálny bod interakcie používateľa s aplikáciou, sprostredkúva prístup ku všetkým funkcionalitám systému a poskytuje vizuálny výstup pre všetky dotazy.

Princíp oddelenia dátovej logiky od GUI

Na rozdiel od dátovej vrstvy, trieda MainFrame **nepracuje priamo** s vnútornými dátovými štruktúrami pacientov a testov. Týmto spôsobom je zabezpečená silná izolácia medzi prezentačnou a logickou vrstvou systému.

1. **Volanie metód systému** - každá akcia používateľa (napr. kliknutie na tlačidlo, výber z menu) je spracovaná v MainFrame a prevedená na volanie verejnej metódy triedy DemoSystem s požadovanými vstupmi.
2. **Vracia sa formátovaný text** – Metódy DemoSystem nevracajú interné objekty (napr. PatientData, PCRTTestDta, ...), ale namiesto toho vracajú už formátované textové reprezentácie vhodné na zobrazenie.
3. **Spracovanie textu** - GUI vrstva prijíma výsledky ako obyčajný text, ktorý sa následne zobrazí používateľovi pomocou komponentov ako **JTextArea** alebo **JOptionPane**. Tento text môže byť (napr. tabuľka výsledkov, sumarizácia testov, chybová alebo potvrdzovacia správa, ...)

GUI teda nezachováva žiadny stav súvisiaci s dátami. Týmto spôsobom je zabezpečená silná izolácia medzi prezentačnou a logickou vrstvou systému.

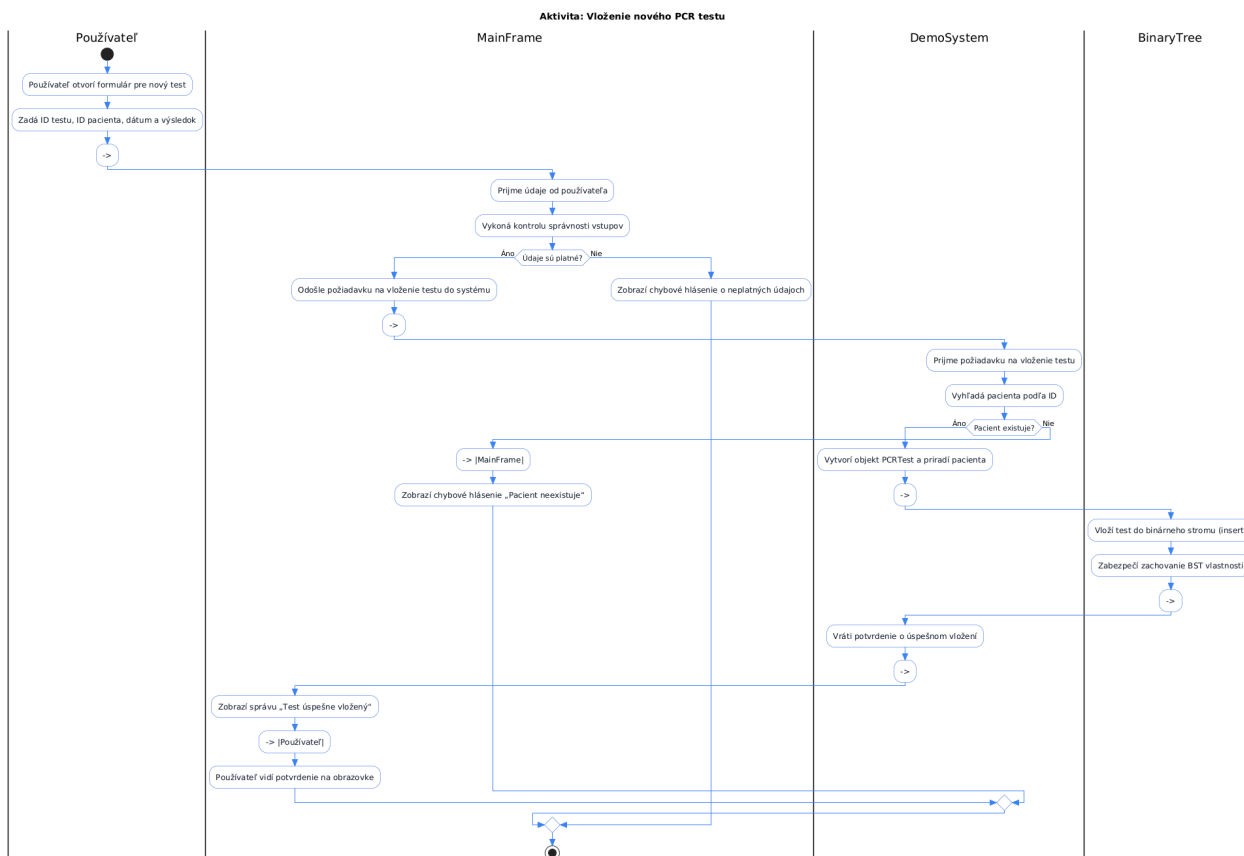
Hlavné funkcionality rozhrania

- **Menu systém:** Načítanie, uloženie a generovanie dát; správa pacientov a testov; pokročilé vyhľadávanie; nápoveda.
- **Rýchle vyhľadávanie:** Hľadať pacienta, test alebo konkrétny test pre pacienta priamo z úvodnej obrazovky.
- **Formuláre pre vloženie dát:** Pridávanie nových pacientov a testov s validáciou vstupov.
- **Pokročilé vyhľadávanie:** Interaktívne dotazy podľa okresov, regiónov, pracovísk, pozitívnych testov, chorých pacientov a zoradené prehľady.
- **Generovanie náhodných dát:** Automatické vytváranie pacientov a testov pre testovanie.
- **Zobrazenie informácií:** Prehľad dátových štruktúr pacientov a testov, vrátane príkladov.

Používané komponenty

- JMenuBar, JMenu, JMenuItem – pre menu systém
- JPanel, JLabel, JTextField, JButton, JSpinner, JTextArea, JScrollPane – pre formuláre a výstupy
- JOptionPane – pre interaktívne dialógy a výpis výsledkov

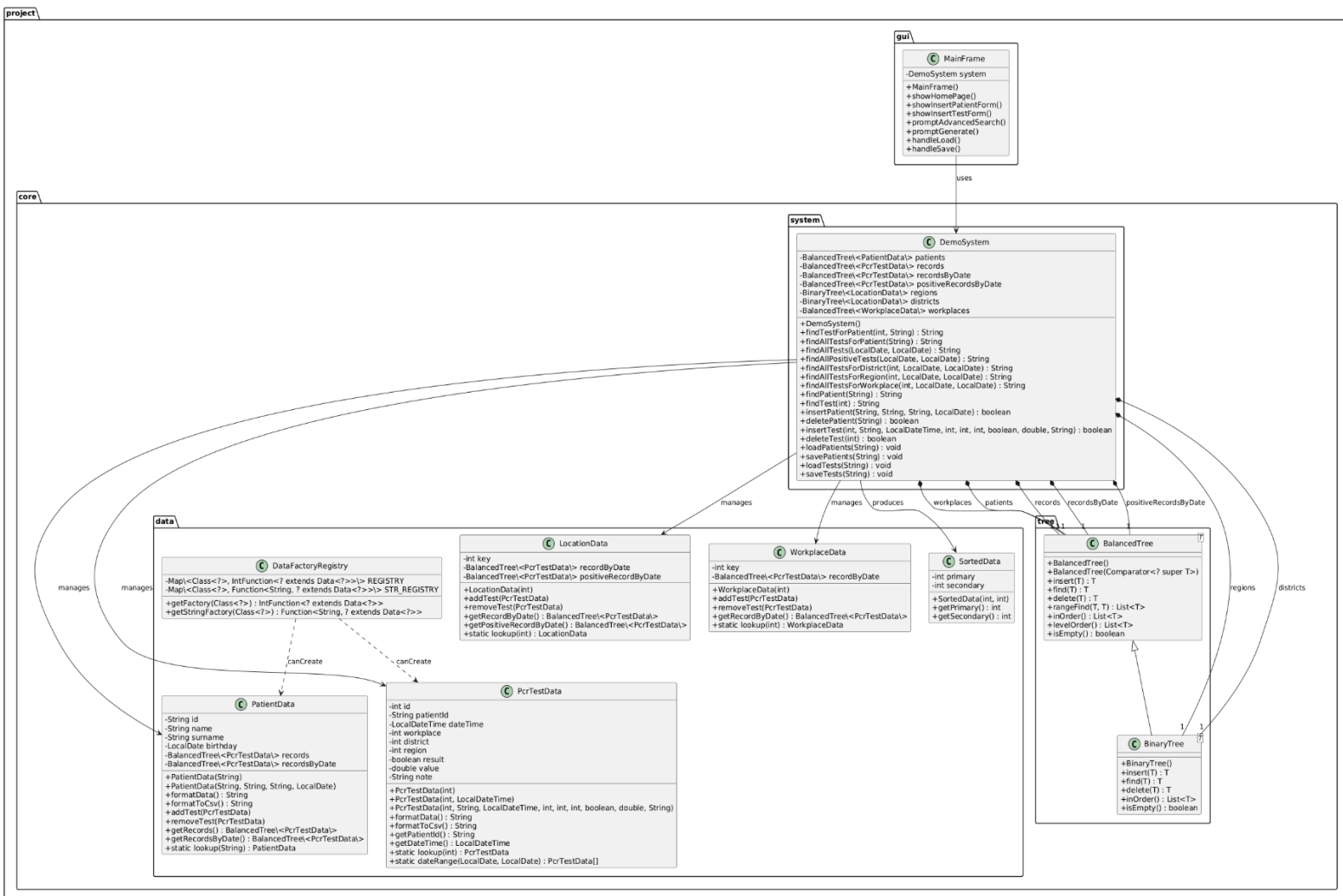
Stručne povedané, MainFrame slúži ako centrálny bod interakcie používateľa so systémom PCR testov, poskytuje jednoduchý prístup ku všetkým funkciám systému a vizualizuje výsledky dotazov.



UML diagram aktivít pre operáciu vloženia testu



UML diagram tried





Využitie AI v projekte

I am having trouble with the rebalancing at the end of delete. My test is showing -> Height mismatch at node 933452590: expected 0, got 1. fix this

Solution — step by step

1. The height mismatch happens because, when deleting a node with two children, the code copies the successor's data into the target node and removes the successor node, but the stack used for rebalancing does not include the nodes between the deleted node and the successor (nor the successor's parent). As a result the rebalancing/height updates start at the wrong place.
2. Fix: after removing the successor, push the path from the deleted node down to the successor's parent onto the stack (so the rebalancing loop will pop and update from the exact node whose child pointer changed up to the root).
3. Make the change inside delete(...) only — push current and the path to successorParent when handling the two-children case. This preserves the rest of the logic.

Adjust the delete(...) method to ensure the rebalancing stack includes the node whose child pointer was changed (successorParent). This pushes current and the path down to successorParent so heights are updated correctly.

```
// Ensure stack contains the path down to the node whose child pointer changed
// (successorParent). The existing stack contains ancestors down to parent of 'current'.
// We must push 'current' and the path from current.right down to successorParent.
if (successorParent == current) {
    // child pointer changed in 'current' itself
    stack.push(current);
} else {
    // push current and the path from current.right to successorParent
    stack.push(current);
    Node<T> n = current.getRight();
    while (n != successorParent && n != null) {
        stack.push(n);
        n = n.getLeft();
    }
    // finally push successorParent (if found)
    if (successorParent != null) {
        stack.push(successorParent);
    }
}
```

can you improve rangeFind()? make it faster or something

Solution — step by step

1. Prune traversal while descending: skip entire left subtrees whose nodes are < min (they cannot contain results).
2. Only push nodes that could be >= min; when popping, stop traversal if node > max.
3. This avoids visiting irrelevant subtrees and reduces work for wide ranges.

Replace rangeFind(...) with a pruned in-order traversal that skips left subtrees that are entirely below min.

in treeCompareTest at the end I need to print out the average time of insert, delete and find for both bst and avl

Solution — step by step

1. Instrument `treeCompareTest` to measure time spent in each operation for `bst` and `avl` separately using `System.nanoTime()`.
2. Keep per-operation accumulators and counters (`insert/delete/find`) for each tree. Count only successful operations (`non-null results`).
3. After the loop compute and print average times (in microseconds) for each operation and each tree.

```
public static <T extends Data<T>> void treeCompareTest(Tree<T> bst, Tree<T> avl, IntFunction<T>
creator) {
```

```
    ThreadLocalRandom rand = ThreadLocalRandom.current();
    ArrayList<T> dataList = new ArrayList<>();
```

```
    long bstInsertTime = 0, avlInsertTime = 0;
    long bstDeleteTime = 0, avlDeleteTime = 0;
    long bstFindTime = 0, avlFindTime = 0;
    int bstInsertCount = 0, avlInsertCount = 0;
    int bstDeleteCount = 0, avlDeleteCount = 0;
    int bstFindCount = 0, avlFindCount = 0;
```

```
    for (int i = 0; i < TOTAL; i++) {
        int operation = rand.nextInt(3);
        switch (operation) {
            case 0 -> {
                T toInsert = creator.apply(rand.nextInt());
                long s = System.nanoTime();
                T resB = null;
                if (bst != null) resB = (T) bst.insert(toInsert);
                long e = System.nanoTime();
                if (resB != null) {
                    bstInsertTime += (e - s);
                    bstInsertCount++;
                }
            }
        }
    }
```

```
    s = System.nanoTime();
    T resA = null;
    if (avl != null) resA = (T) avl.insert(toInsert);
    e = System.nanoTime();
    if (resA != null) {
        avlInsertTime += (e - s);
        avlInsertCount++;
    }
}
```

```
// keep only values that succeeded in both trees for later deletes
if (resB != null && resA != null) {
```



```
        dataList.add(toInsert);
    } else {
        // if one succeeded and other failed, best-effort cleanup:
        if (resB != null && resA == null) bst.delete(toInsert);
        if (resA != null && resB == null) avl.delete(toInsert);
    }
}
}
case 1 -> {
    if (!dataList.isEmpty()) {
        T toDelete = dataList.get(rand.nextInt(dataList.size()));

        long s = System.nanoTime();
        Object drB = bst.delete(toDelete);
        long e = System.nanoTime();
        if (drB != null) {
            bstDeleteTime += (e - s);
            bstDeleteCount++;
        }

        s = System.nanoTime();
        Object drA = avl.delete(toDelete);
        e = System.nanoTime();
        if (drA != null) {
            avlDeleteTime += (e - s);
            avlDeleteCount++;
        }
    }

    dataList.remove(toDelete);
}
}
case 2 -> {
    if (!dataList.isEmpty()) {
        T toFind = dataList.get(rand.nextInt(dataList.size()));

        long s = System.nanoTime();
        Object frB = bst.find(toFind);
        long e = System.nanoTime();
        if (frB != null) {
            bstFindTime += (e - s);
            bstFindCount++;
        }

        s = System.nanoTime();
        Object frA = avl.find(toFind);
        e = System.nanoTime();
        if (frA != null) {
            avlFindTime += (e - s);
            avlFindCount++;
        }
    }
}
```



```

    }
    }
    }
    }
}

// compute averages in microseconds
double bstInsertAvg = bstInsertCount == 0 ? 0 : bstInsertTime / (double) bstInsertCount /
1_000.0;
double avlInsertAvg = avlInsertCount == 0 ? 0 : avlInsertTime / (double) avlInsertCount / 1_000.0;
double bstDeleteAvg = bstDeleteCount == 0 ? 0 : bstDeleteTime / (double) bstDeleteCount /
1_000.0;
double avlDeleteAvg = avlDeleteCount == 0 ? 0 : avlDeleteTime / (double) avlDeleteCount /
1_000.0;
double bstFindAvg = bstFindCount == 0 ? 0 : bstFindTime / (double) bstFindCount / 1_000.0;
double avlFindAvg = avlFindCount == 0 ? 0 : avlFindTime / (double) avlFindCount / 1_000.0;

System.out.println("\nAverage times (μs):");
System.out.printf("BST - insert: %.3f delete: %.3f find: %.3f%n", bstInsertAvg, bstDeleteAvg,
bstFindAvg);
System.out.printf("AVL - insert: %.3f delete: %.3f find: %.3f%n", avlInsertAvg, avlDeleteAvg,
avlFindAvg);

System.out.println("\nAll basic test completed in " + bst.getClass().getSimpleName() + "\n");
}

```

I need to compare my avl tree against another data structure in insert, delete and find. Which structure in java can be suitable?

Best match for your AVL tree.

- Internally implemented as a Red-Black Tree (a self-balancing BST, similar complexity to AVL).
- Provides $O(\log n)$ for insert, delete, and lookup.
- Maintains sorted order automatically.
- Perfect for direct comparison since both are balanced BSTs.

Recommendation: Use `TreeSet<T>` if you only care about keys (like your `IntData`).

Almost identical complexity characteristics to AVL.

Balanced tree, so behavior matches well.

Very stable, battle-tested implementation.

import java.util.function.IntFunction; IntFunction<IntData> creator = IntData::new; // no reflection for (int i = 0; i < 10_000_000; i++) { tree.insert(creator.apply(rand.nextInt())); } this looks interesting. How could I do this in my code then provided I only have acces to the this variable BinaryTree<IntData> bst = new BinaryTree<>();? It's root is initialized with a cored node containing a corect data type value



Excellent — you're thinking in exactly the right direction.

Let's walk through how to turn that runtime `BinaryTree<IntData> bst` (with a root containing an `IntData`) into a fast, reusable data creator using lambdas — without knowing the type ahead of time.

```
import java.util.*;
import java.util.concurrent.ThreadLocalRandom;
import java.util.function.IntFunction;

public class DataFactoryRegistry {
    private static final Map<Class<?>, IntFunction<? extends Data<?>>> FACTORIES = new
    HashMap<>();
    static {
        FACTORIES.put(IntData.class, IntData::new);
        FACTORIES.put(StringData.class, i -> new StringData("S" + i));
    }
    @SuppressWarnings("unchecked")
    public static <T extends Data<T>> IntFunction<T> getFactory(Class<?> clazz) {
        return (IntFunction<T>) FACTORIES.get(clazz);
    }
}
```

For each public endpoint in DemoSystem I want to enable acces trough the mainframe gui.

Solution — step by step

1. Wire MainFrame menu actions to call DemoSystem public methods.
2. Use simple input dialogs (JOptionPane) to collect parameters and show results in a scrollable text area.
3. Add file load/save via JFileChooser for patients/tests.
4. Keep UI changes minimal and centralized in MainFrame; reuse existing PatientPanel for new-patient entry (unchanged).
5. Validate/parsing errors produce error dialogs.

contentPanel.add(right, BorderLayout.EAST); Keep just this in the welcome page. Dont change the quick search just make it so taht after searching the search text field gets set to empty, so the seached id does not persisit. Also ad some more tips

Solution — step by step

1. Update MainFrame.showHomePage() to clear the search text fields after a successful search (patientSearch and testSearch).
2. Add more tips to the status area text.
3. Keep the welcome page only adding the right panel (contentPanel.add(right, BorderLayout.EAST)) — no other additions.

When inputing the DateTime for test and birthday for patient insted of text field editor create some form of calendar where the user can select the date and time. doth to to the hour and minute

Solution — step by step



1. Replace the birthday and test DateTime text fields with JSpinner controls (SpinnerDateModel) that show a calendar-like date/time editor.
2. Use JSpinner.DateEditor to format: birthday spinner uses "yyyy-MM-dd", test spinner uses "yyyy-MM-dd'T'HH:mm" (hour:minute).
3. Convert spinner Date values to LocalDate / LocalDateTime using Instant + ZoneId when inserting.
4. Minimal, local changes only in showInsertPatientForm() and showInsertTestForm(); keep validation and existing logic.

In promptAdvancedSearch() make the date time selection through the same editor during test insertion, meaning through the JSpinner.DateEditor

Solution — step by step

1. Replace the plain text prompts for date/time in promptAdvancedSearch() with a reusable JSpinner-based date/time picker (same format as test insertion).
2. Add a small helper promptForDateTime(title) that shows a JOptionPane with a JSpinner (DateEditor "yyyy-MM-dd HH:mm") and returns a LocalDateTime or null if cancelled.
3. Use promptForDateTime in all places in promptAdvancedSearch where a date/time was requested.

MainFrame.java

Update promptAdvancedSearch(...) to use the spinner editor and add promptForDateTime(...). Only changed/added regions shown.

```
```java
// ...existing code...
private void promptAdvancedSearch() {
 String[] options = {
 "All tests (date range)",
 "All positive tests (date range)",
 "Tests for district (date range)",
 "Tests for region (date range)",
 "Tests for workplace (date range)",
 "Sick patients for district (date range)",
 "Most sick patient per district (date range)",
 "Sorted districts by sick patients (date range)"
 };
 String choice = (String) JOptionPane.showInputDialog(this, "Select search", "Advanced Search",
 JOptionPane.PLAIN_MESSAGE, null, options, options[0]);
 if (choice == null) return;
 try {
 switch (choice) {
 case "All tests (date range)" -> {
 - LocalDateTime min = LocalDateTime.parse(JOptionPane.showInputDialog(this, "Min
dateTime:"));
 - LocalDateTime max = LocalDateTime.parse(JOptionPane.showInputDialog(this, "Max
dateTime:"));
 }
 }
 }
}
```



```
+ LocalDateTime min = promptForDateTime("Min dateTime:");
+ if (min == null) return;
+ LocalDateTime max = promptForDateTime("Max dateTime:");
+ if (max == null) return;
+ showLongText("All tests", system.findAllTests(min, max));
 }
 case "All positive tests (date range)" -> {
- LocalDateTime min = LocalDateTime.parse(JOptionPane.showInputDialog(this, "Min
dateTime:"));
- LocalDateTime max = LocalDateTime.parse(JOptionPane.showInputDialog(this, "Max
dateTime:"));
+ LocalDateTime min = promptForDateTime("Min dateTime:");
+ if (min == null) return;
+ LocalDateTime max = promptForDateTime("Max dateTime:");
+ if (max == null) return;
+ showLongText("Positive tests", system.findAllPositiveTests(min, max));
 }
 case "Tests for district (date range)" -> {
 int district = Integer.parseInt(JOptionPane.showInputDialog(this, "District code:"));
- LocalDateTime min = LocalDateTime.parse(JOptionPane.showInputDialog(this, "Min
dateTime:"));
- LocalDateTime max = LocalDateTime.parse(JOptionPane.showInputDialog(this, "Max
dateTime:"));
+ LocalDateTime min = promptForDateTime("Min dateTime:");
+ if (min == null) return;
+ LocalDateTime max = promptForDateTime("Max dateTime:");
+ if (max == null) return;
+ showLongText("Tests for district " + district, system.findAllTestsForDistrict(district, min,
max));
 }
 case "Tests for region (date range)" -> {
 int region = Integer.parseInt(JOptionPane.showInputDialog(this, "Region code:"));
- LocalDateTime min = LocalDateTime.parse(JOptionPane.showInputDialog(this, "Min
dateTime:"));
- LocalDateTime max = LocalDateTime.parse(JOptionPane.showInputDialog(this, "Max
dateTime:"));
+ LocalDateTime min = promptForDateTime("Min dateTime:");
+ if (min == null) return;
+ LocalDateTime max = promptForDateTime("Max dateTime:");
+ if (max == null) return;
+ showLongText("Tests for region " + region, system.findAllTestsForRegion(region, min,
max));
 }
 case "Tests for workplace (date range)" -> {
 int wp = Integer.parseInt(JOptionPane.showInputDialog(this, "Workplace code:"));
- LocalDateTime min = LocalDateTime.parse(JOptionPane.showInputDialog(this, "Min
dateTime:"));
```



```
- LocalDateTime max = LocalDateTime.parse(JOptionPane.showInputDialog(this, "Max
dateTime:"));
+ LocalDateTime min = promptForDateTime("Min dateTime:");
+ if (min == null) return;
+ LocalDateTime max = promptForDateTime("Max dateTime:");
+ if (max == null) return;
 showLongText("Workplace tests", system.findAllTestsForWorkplace(wp, min, max));
 }
 case "Sick patients for district (date range)" -> {
 int district = Integer.parseInt(JOptionPane.showInputDialog(this, "District code:"));
- LocalDateTime date = LocalDateTime.parse(JOptionPane.showInputDialog(this,
"Reference date (end):"));
+ LocalDateTime date = promptForDateTime("Reference date (end):");
+ if (date == null) return;
 int days = Integer.parseInt(JOptionPane.showInputDialog(this, "Days back:"));
 showLongText("Sick patients", system.findAllSickPatientsForDistrict(district, date, days));
 }
 case "Most sick patient per district (date range)" -> {
- LocalDateTime date = LocalDateTime.parse(JOptionPane.showInputDialog(this,
"Reference date (end):"));
+ LocalDateTime date = promptForDateTime("Reference date (end):");
+ if (date == null) return;
 int days = Integer.parseInt(JOptionPane.showInputDialog(this, "Days back:"));
 showLongText("Most sick patients", system.findMostSickPatientForDistrict(date, days));
 }
 case "Sorted districts by sick patients (date range)" -> {
- LocalDateTime date = LocalDateTime.parse(JOptionPane.showInputDialog(this,
"Reference date (end):"));
+ LocalDateTime date = promptForDateTime("Reference date (end):");
+ if (date == null) return;
 int days = Integer.parseInt(JOptionPane.showInputDialog(this, "Days back:"));
 showLongText("Sorted districts", system.findSortedDistrictsBySickPatients(date, days));
 }
 default -> JOptionPane.showMessageDialog(this, "Not implemented", "Info",
JOptionPane.INFORMATION_MESSAGE);
 }
 } catch (DateTimeParseException | NumberFormatException ex) {
 JOptionPane.showMessageDialog(this, "Invalid input: " + ex.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);
 }
}

// helper: show a date-time picker dialog using JSpinner.DateEditor, returns null on cancel
private java.time.LocalDateTime promptForDateTime(String title) {
 SpinnerDateModel model = new SpinnerDateModel(new Date(), null, null,
java.util.Calendar.MINUTE);
 JSpinner spinner = new JSpinner(model);
 JSpinner.DateEditor editor = new JSpinner.DateEditor(spinner, "yyyy-MM-dd HH:mm");
```



```
spinner.setEditor(editor);
spinner.setPreferredSize(new Dimension(220, 30));

int res = JOptionPane.showConfirmDialog(this, spinner, title, JOptionPane.OK_CANCEL_OPTION,
JOptionPane.PLAIN_MESSAGE);
if (res != JOptionPane.OK_OPTION) {
 return null;
}
Date selected = (Date) spinner.getValue();
return
Instant.ofEpochMilli(selected.getTime()).atZone(ZoneId.systemDefault()).toLocalDateTime();
}

// ...existing code...
```